Get started Open in app

towards
data science

Follow 569K Followers

You have **2** free member-only stories left this month. Sign up for Medium and get an extra one

Data Structures & Algorithms in Python

An overview to start mastering data structures and algorithms

Papa Moryba Kouate Aug 14, 2020 · 8 min read ★

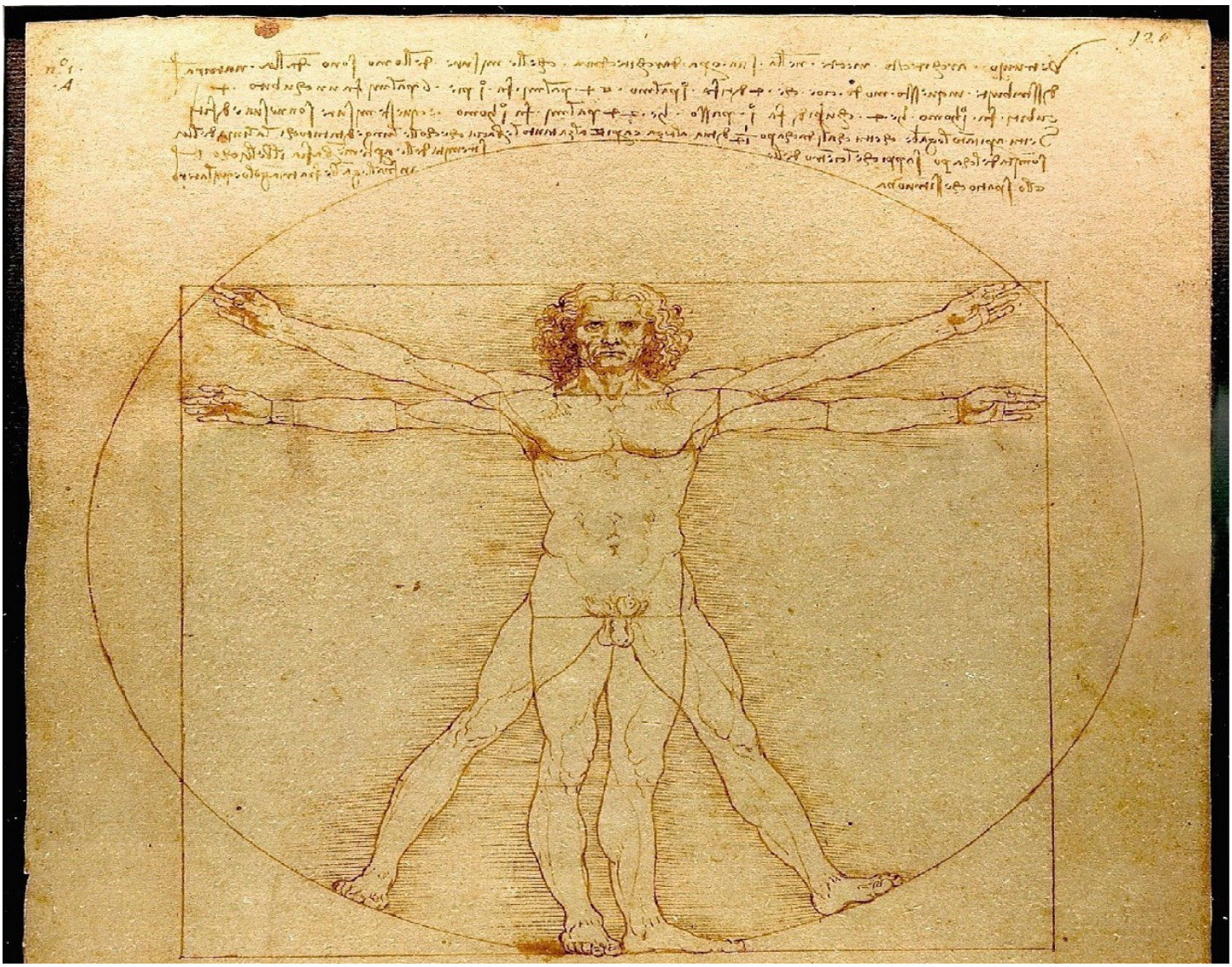

Image by WikiImages from Pixabay

> To develop a complete mind: Study the science of art; Study the art of science. Learn how to see. Realize that everything connects to everything else.
>
> Leonardo da Vinci

. . .

Introduction

The purpose of this article is to give you a panorama of data structures and algorithms in Python. This topic is very important for a Data Scientist in order to help him or her to design and solve machine learning models in a more effective way.

We will see together with practical examples the built-in data structures, the user-defined data structures, and last but not least I will introduce you to some algorithms like traversal algorithms, sorting algorithms, and searching algorithms.

So, let's get started!

. . .

Part I: Bult-in Data Structures

As the name suggests, data structures allow us to organize, store, and manage data for efficient access and modification.

In this part, we are going to take a look at built-in data structures. There are four types of built-in data structures in Python: list, tuple, set, and dictionary.

List

A list is defined using square brackets and holds data that is separated by commas. The list is mutable and ordered. It can contain a mix of different data types.

```
months = ['january','february','march','april','may','june','july','august','september',
```

```python
print(months[0]) # print the elemment with index 0
print(months[0:7]) # all the elements from index 0 to 6.
months[0] = 'birthday' # exchange the value in index 0 with the word birthday
print(months)
```

example_list.py hosted with ❤ by GitHub view raw

out:

```
january

['january', 'february', 'march', 'april', 'may', 'june', 'july']

['birthday', 'february', 'march', 'april', 'may', 'june', 'july',
'august', 'september', 'october', 'november', 'december']
```

Below there are some useful functions for the list.

```python
# Join is a string method that takes a list of strings as an argument,
# and returns a string consisting of the list elements joined by a separator string.

first_str = "\n".join(["What","is","your","favourite","painting","?"])
second_str = "-".join(["Who","is","your","favourite","artist","?"])

print(first_str)
print(second_str)
```

join.py hosted with ❤ by GitHub view raw

out:

```
What
is
your
favourite
painting
?

Who-is-your-favourite-artist-?
```

```python
artist = ['Chagall', 'Kandinskij', 'Dalí', 'da Vinci', 'Picasso', 'Warhol']

artist.append('Basquiat') # append method permits us to add Basquiat in our list of arti
```

```
artist.append('Basquiat') # append method permits us to add Basquiat in our list of arti
print(artist)
```

append.py hosted with ❤ by GitHub view raw

out:

```
['Chagall', 'Kandinskij', 'Dalí', 'da Vinci', 'Picasso', 'Warhol',
'Basquiat']
```

Tuple

A tuple is another container. It is a data type for immutable ordered sequences of elements. Immutable because you can't add and remove elements from tuples, or sort them in place.

```
length, width, height = 7, 3, 1 # we can assign multiple varibles in one shot
print("The dimensions are {} x {} x {}".format(length, width, height))
```

tuples.py hosted with ❤ by GitHub view raw

out:

```
The dimensions are 7 x 3 x 1
```

Set

Set is a mutable and unordered collection of unique elements. It can permit us to remove duplicate quickly from a list.

```
numbers = [1, 2, 6, 3, 1, 1, 5]

unique_nums = set(numbers)
print(unique_nums)


artist = {'Chagall', 'Kandinskij', 'Dalí', 'da Vinci', 'Picasso', 'Warhol', 'Basquiat'}

print('Turner' in artist) # check if there is Turner in the set artist
artist.add('Turner')
```

```
print(artist.pop()) #remove the last item
```

set.py hosted with ❤ by GitHub view raw

out:

```
{1, 2, 3, 5, 6}
False
Basquiat
```

Dictionary

Dictionary is a mutable and unordered data structure. It permits storing a pair of items (i.e. keys and values).

As the example below shows, in the dictionary, it is possible to include containers into other containers to create compound data structures.

```
music = { 'jazz': {"Coltrane": "In a Sentimental Mood",
                           "M.Davis":"Blue in Green" ,
                           "T.Monk":"Don't Blame Me"},
             "classical" : {"Bach": "Cello Suit",
                         "Mozart": "Lacrimosa",
                         "Satie": "Gymnopédie"}}

print(music["jazz"]["Coltrane"]) # we select the value of the key Coltrane
print(music["classical"]["Mozart"])
```

dictionary.py hosted with ❤ by GitHub view raw

out:

```
In a Sentimental Mood
Lacrimosa
```

. . .

Part II: User-Defined Data Structures

Now I will introduce you three user-defined data structures: ques, stack, and tree. I assume that you have a basic knowledge of classes and functions.

Stack using arrays

The stack is a linear data structure where elements are arranged sequentially. It follows the mechanism L.I.F.O which means last in first out. So, the last element inserted will be removed as the first. The operations are:

- Push → inserting an element into the stack
- Pop → deleting an element from the stack

The conditions to check:

- overflow condition → this condition occurs when we try to put one more element into a stack that is already having maximum elements.
- underflow condition →this condition occurs when we try to delete an element from an empty stack.

```
class mystack:

    def __init__(self):
        self.data =[]

    def length(self): #length of the list
        return len(self.data)

    def is_full(self): #check if the list is full or not
        if len(self.data) == 5:
            return True
        else:
            return False

    def push(self, element):# insert a new element
        if len(self.data) < 5:
            self.data.append(element)
        else:
            return "overflow"

    def pop(self): # # remove the last element from a list
        if len(self.data) == 0:
            return "underflow"
        else:
```

```
            return self.data.pop()

a = mystack() # I create my object
a.push(10) # insert the  element
a.push(23)
a.push(25)
a.push(27)
a.push(11)
print(a.length())
print(a.is_full())
print(a.data)
print(a.push(31)) # we try to insert one more element in the list - the output will be
print(a.pop())
print(a.pop())
print(a.pop())
print(a.pop())
print(a.pop())
print(a.pop()) # try to delete an element in a list without elements - the output will

```

stack.py hosted with ❤ by GitHub view raw

out:

```
5
True
[10, 23, 25, 27, 11]
overflow
11
27
25
23
10
underflow
```

Queue using arrays

The queue is a linear data structure where elements are in a sequential manner. It follows the F.I.F.O mechanism that means first in first out. Think when you go to the cinema with your friends, as you can imagine the first of you that give the ticket is also the first that step out of the line. The mechanism of the queue is the same.

Below the aspects that characterize a queue.

Two ends:

- front → points to starting element
- rear → points to the last element

There are two operations:

- enqueue → inserting an element into the queue. It will be done at the rear.
- dequeue → deleting an element from the queue. It will be done at the front.

There are two conditions:

- overflow → insertion into a queue that is full
- underflow → deletion from the empty queue

```
class myqueue:

    def __init__(self):
        self.data = []

    def length(self):
        return len(self.data)

    def enque(self, element): # put the element in the queue
        if len(self.data) < 5:
            return self.data.append(element)
        else:
            return "overflow"

    def deque(self): # remove the first element that we have put in queue
        if len(self.data) == 0:
             return "underflow"
        else:
            self.data.pop(0)

b = myqueue()
b.enque(2) # put the element into the queue
b.enque(3)
b.enque(4)
b.enque(5)
print(b.data)
b.deque()# # remove the first element that we have put in the queue
print(b.data)
```

queue.py hosted with ❤ by GitHub view raw

out:

```
[2, 3, 4, 5]
[3, 4, 5]
```

Tree (general tree)

Trees are used to define hierarchy. It starts with the root node and goes further down, the last nodes are called child nodes.

In this article, I focus on the binary tree. The binary tree is a tree data structure in which each node has at most two children, which are referred to as the left child and the right child. Below you can see a representation and an example of the binary tree with python where I constructed a class called **Node** and the objects that represent the different nodes(A, B, C, D, and E).

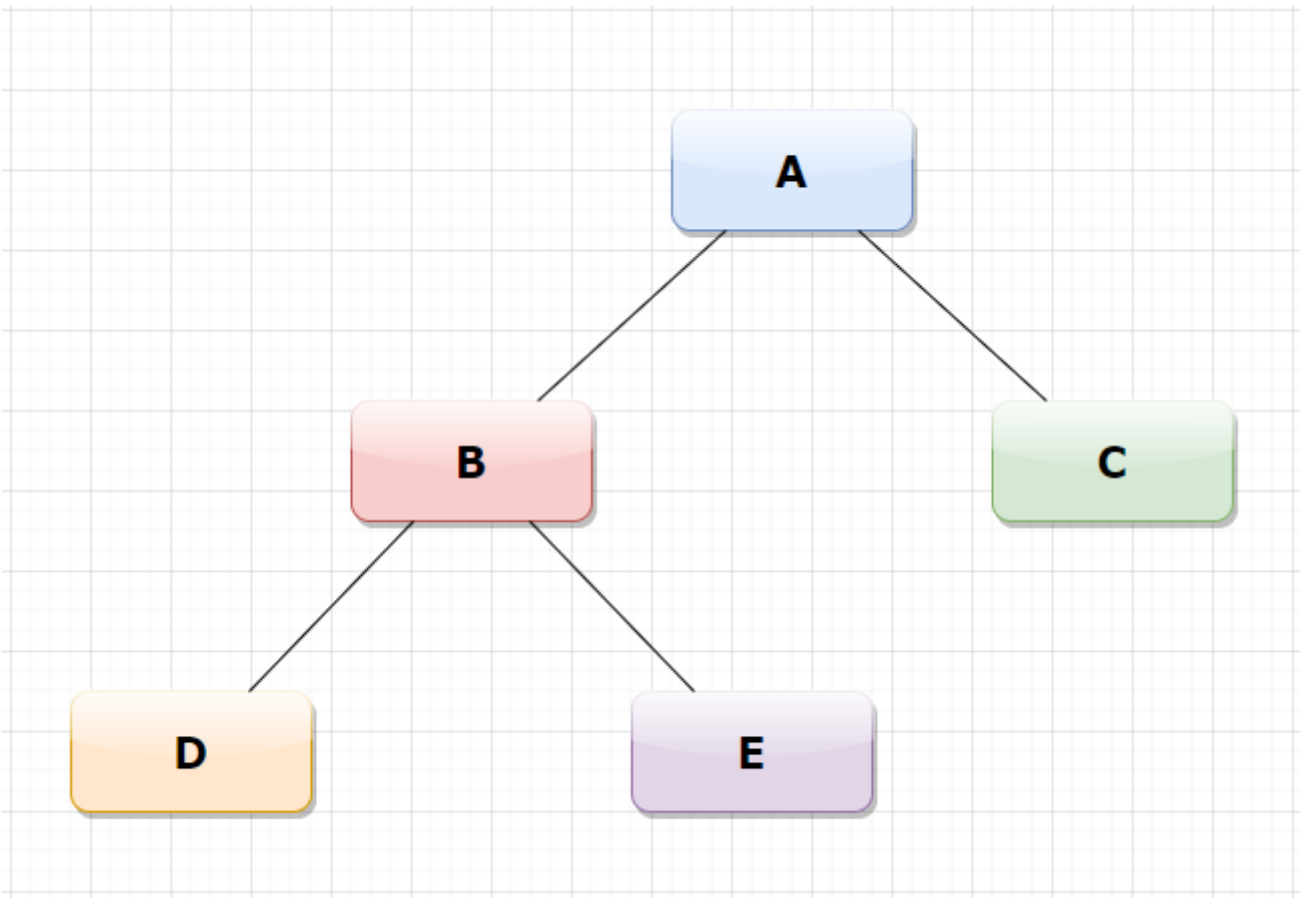

image by author

```
# create the class Node and the attrbutes
```

```python
# create the class Node and the attributes
class Node:
    def __init__(self, letter):
        self.childleft = None
        self.childright = None
        self.nodedata = letter

# create the nodes for the tree
root = Node('A')
root.childleft = Node('B')
root.childright = Node('C')
root.childleft.childleft = Node('D')
root.childleft.childright = Node('E')
```

class.py hosted with ❤ by GitHub view raw

Anyway, there are other user-defined data structures like **linked lists** and **graphs.**

. . .

Part III: Algorithms

The concept of the algorithm has existed since antiquity. In fact, the ancient Egyptians used algorithms to solve their problems. Then they taught this approach to the Greeks.

The word algorithm derives itself from the 9th-century Persian mathematician Muḥammad ibn Mūsā al-Khwārizmī, whose name was Latinized as Algorithmi. Al-Khwārizmī was also an astronomer, geographer, and a scholar in the House of Wisdom in Baghdad.

As you already know algorithms are instructions that are formulated in a finite and sequential order to solve problems.

When we write an algorithm, we have to know what is the exact problem, determine where we need to start and stop and formulate the intermediate steps.

There are three main approaches to solve algorithms:

- Divide et Impera (also known as divide and conquer) → it divides the problem into sub-parts and solves each one separately
- Dynamic programming → it divides the problem into sub-parts remembers the results of the sub-parts and applies it to similar ones

- Greedy algorithms → involve taking the easiest step while solving a problem without worrying about the complexity of the future steps

Tree Traversal Algorithm

Trees in python are non-linear data structures. They are characterized by roots and nodes. I take the class I constructed before for the binary tree.

Tree Traversal refers to visiting each node present in the tree exactly once, in order to update or check them.

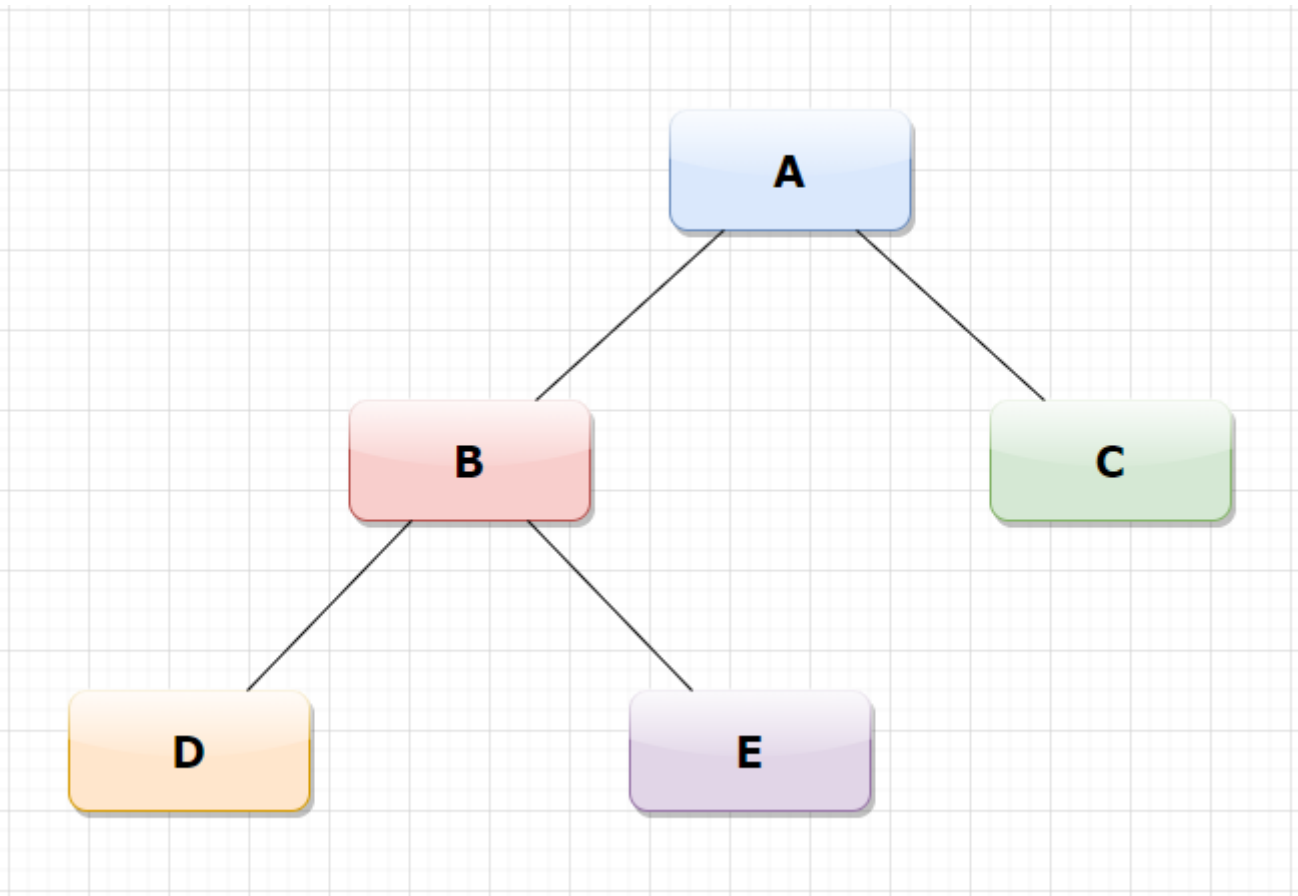

image by author

```
# create the class Node and the attrbutes
class Node:
    def __init__(self, letter):
        self.childleft = None
        self.childright = None
        self.nodedata = letter

# create the nodes for the tree
root = Node('A')
root.childleft = Node('B')
root.childright = Node('C')
```

```
root.childleft.childleft = Node('D')
root.childleft.childright = Node('E')
```

class.py hosted with ❤ by GitHub view raw

There are three types of tree traversals:

- In-order traversal → refers to visiting the left node, followed by the root and then the right nodes.

Here D is the leftmost node where the nearest root is B. The right of root B is E. Now the left sub-tree is completed, so I move towards the root node A and then to node C.

```
def InOrd(root):
    if root:
        InOrd(root.childleft)
        print(root.nodedata)
        InOrd(root.childright)
InOrd(root)
```

InOrd.py hosted with ❤ by GitHub view raw

out:

```
D
B
E
A
C
```

- Pre-order traversal → refers to visiting the root node followed by the left nodes and then the right nodes.

In this case, I move to the root node A and then to the left child node B and to the sub child node D. After that I can go to the nodes E and then C.

```
def PreOrd(root):
    if root:
        print(root.nodedata)
        PreOrd(root.childleft)
        PreOrd(root.childright)
PreOrd(root)
```

PreOrd.py hosted with ❤ by GitHub view raw

out:

```
A
B
D
E
C
```

- Post-order traversal → refers to visiting the left nodes followed by the right nodes and then the root node

I go to the most left node which is D and then to the right node E. Then, I can go from the left node B to the right node C. Finally, I move towards the root node A.

```python
def PostOrd(root):
    if root:
        PostOrd(root.childleft)
        PostOrd(root.childright)
        print(root.nodedata)
PostOrd(root)
```

PostOrd.py hosted with ❤ by GitHub view raw

out:

```
D
E
B
C
A
```

Sorting Algorithm

The sorting algorithm is used to sort data in some given order. It can be classified in Merge Sort and Bubble Sort.

- Merge Sort → it follows the **divide et Impera rule**. The given list is first divided into smaller lists and compares adjacent lists and then, reorders them in the desired

sequence. So, in summary from unordered elements as input, we need to have ordered elements as output. Below, the code with each step described.

```
def merge_sort(ourlist, left, right): #left and right corresponds to starting and endir
    if right -left > 1: # check if the length of ourlist is greater than 1
        middle = (left + right) // 2 # we divide the length in two parts
        merge_sort(ourlist, left, middle) # recursevely I call the merge_sort function
        merge_sort(ourlist, middle, right) # then from middle to right
        merge_list(ourlist, left, middle, right) # finally I create ourlist in complete

def merge_list(ourlist, left, middle, right):# I create the function merged_list
    leftlist = ourlist[left:middle] # I define the leftlist
    rightlist = ourlist[middle:right] # I define the right list
    k = left # it is the the temporary variable
    i = 0 # this variable that corespond to the index of the first group help me to ite
    j = 0 # this variable that corespond to the index of the second group help me to it
    while (left + i < middle and middle+ j < right): # the condition that I want to ach
        if (leftlist[i] <= rightlist[j]): #if the element in the leftlist is less or ec
            ourlist[k] = leftlist[i] # In this case I fill the value of the leftlist ir
            i = i + 1 #now I have to increment the value by 1
        else: # if the above codition is not match
            ourlist[k] = rightlist[j] # I fill the rightlist element in ourlist with ir
            j = j + 1 # I increment index j by 1
        k = k+1 # now I increment the value of k by 1
    if left + i < middle: # check if left + i is less than middle
        ourlist[k] = leftlist[i] # I place all elements of my leftlist in ourlist
        i = i + 1
        k = k + 1
    else: # otherwise if my leftlist is empty
        while k < right: # untill k is less then right
            ourlist[k] = rightlist[j] # I place all elements of rightlist in ourlist
            j = j + 1
            k = k + 1

ourlist = input('input - unordered elements: ').split() # insert the input and split
ourlist = [int(x) for x in ourlist]
merge_sort(ourlist, 0, len(ourlist))
print('output - ordered elements: ')
print(ourlist)
```

mergesort.py hosted with ❤ by GitHub view raw

out:

```
input - unordered elements: 15 1 19 93
output - ordered elements:
[1, 15, 19, 93]
```

- Bubble Sort → it first compares and then sorts adjacent elements if they are not in the specified order.

```
def bubble_sort(ourlist): # I create my function bubble_sort with the argument called o
    b=len(ourlist)-1 # for every list, I will have a minus 1 iteration
    for x in range(b): # for each element in the range of b, I check if they are ordere
        for y in range(b-x):
            if ourlist[y]>ourlist[y+1]: # if one element is greater than the nearest el
                ourlist[y],ourlist[y+1]=ourlist[y+1],ourlist[y] # I have to swap the el
                                                                # I exchange the position of
        return ourlist

ourlist=[15,1,9,3]
bubble_sort(ourlist)
```

bubble.py hosted with ❤ by GitHub view raw

out:

```
[1, 3, 9, 15]
```

- Insertion Sort → it picks one item of a given list at the time and places it at the exact spot where it is to be placed.

```
def bubble_sort(ourlist): # I create my function bubble_sort with the argument called o
    b=len(ourlist)-1 # for every list, I will have a minus 1 iteration
    for x in range(b): # for each element in the range of b, I check if they are ordere
        for y in range(b-x):
            if ourlist[y]>ourlist[y+1]: # if one element is greater than the nearest el
                ourlist[y],ourlist[y+1]=ourlist[y+1],ourlist[y] # I have to swap the el
                                                                # I exchange the position of
        return ourlist

ourlist=[15,1,9,3]
bubble_sort(ourlist)
```

bubble.py hosted with ❤ by GitHub view raw

out:

```
[1, 3, 9, 15]
```

There are other Sorting Algorithms like **Selection Sort** and **Shell Sort**.

Searching Algorithms

Searching algorithms are used to seek for some elements present in a given dataset. There are many types of search algorithms such as Linear Search, Binary Search, Exponential Search, Interpolation Search, and so on. In this section, we will see the Linear Search and Binary Search.

- Linear Search → in a single-dimensional array we have to search a particular key element. The input is the group of elements and the key element that we want to find. So, we have to compare the key element with each element of the group. In the following code, I try to seek element 27 in our list.

```python
def lin_search(ourlist, key):

    for index in range(0, len(ourlist)):
        if (ourlist[index] == key):
            return  index
    else:
        return "not fund"

ourlist = [15, 1, 9, 3]

lin_search(ourlist, 27)
```

linsearch.py hosted with ❤ by GitHub view raw

out:

```
'not fund'
```

- Binary Search → in this algorithm, we assume that the list is in ascending order. So, if the value of the search key is less than the element in the middle of the list, we narrow the interval to the lower half. Otherwise, we narrow to the upper half. We continue our check until the value is found or the list is empty.

```
def bin_search(ourlist, key):
    left = 0 # I assign left position to zero
    right = len(ourlist)-1 # I assign right position by defining the length of ourlist
    matched = False
    while(left<=right and not matched): # the loop will continue untill the left elemer
        mid = (left+right)//2 # I find the position of the middle element
        if ourlist[mid] == key: # if the middle element correponds to the key element
             matched = True
        else: #otherwise
            if key < ourlist[mid]: # if key element is less than the middle element
                right = mid - 1 #I assign the position of the right element as mid - 1
            else: #otherwise
                left = mid + 1 #left position will become the middle position plus 1
    return matched

print(bin_search([1, 3, 9, 15], 17))
print(bin_search([1, 3, 9, 15], 3))
```

binary.py hosted with ❤ by GitHub view raw

out:

```
False
True
```

. . .

Conclusion

Now you have an overview of data structures and algorithms. So, you can start going to a deeper understanding of algorithms.

The beautiful image of the Vitruvian Man I have chosen for this article is not casual. The drawing is based on the correlation of the ideal human body in relation to geometry. In fact, for this representation, Leonardo da Vinci was inspired by Vitruvius

who described the man's body as the ideal body to determine the correct proportion in architecture.

For what concerns algorithms, the Vitruvian Man hides a secret algorithm used by the artists for centuries to certify that their works were inspired by the divine proportion.

Sometimes I like to think that maybe Leonardo da Vinci, through his wonderful works, wanted to define the most important algorithm which is the algorithm of life.

. . .

Thanks for reading this. There are some other ways you can keep in touch with me and follow my work:

- Subscribe to my newsletter.
- You can also get in touch via my Telegram group, *Data Science for Beginners*.

Thanks to Anne Bonner.

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

Your email

Get this newsletter

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Machine Learning | Data Science | Programming | Towards Data Science | Python

Medium

About Help Legal

Get the Medium app

Download on the App Store

GET IT ON Google Play